Takam Search

חיפוש טוב יותר בהוראות תכ"ם: הבעיה, הפתרון והארכיטקטורה

נתן מסיקה · 11.11.2026 האתר: `takam-search.vercel.app` הקוד: `github.com/nathmsi/takam-search`

1 הבעיות שמצאתי באתר תכ"ם

אתר הוראות תכ"ם (`takam.mof.gov.il`) הוא המקום שבו עובדי משרדי הממשלה מחפשים את ההנחיות המחייבות של החשב הכללי. השתמשתי באתר, קראתי את הקוד שלו וקראתי ל-API הציבורי שלו (1,126 מסמכים, 114 פרקים). אלה הממצאים:

- **החיפוש לא מוצא:** יחיד ורבים הן שתי מילים שונות, שאלה לא מחזירה כלום (הבעיה המרכזית, בפרק הבא).
- **דף שנשבר כשאין תוצאות:** הכתובת הופכת ל- `/document/undefined` וחלונית הפרטים נשארת ריקה, בלי שום עזרה.
- **דגל התוקף הפוך:** `isvalid` הוא `true` דווקא ב-51 מסמכים שתוקפם פג, והאתר מסתיר זאת בקוד (`isvalid == 1 ? "כן" : "לא"`).
- **פרק שלם חסר בסינון:** פרק 10, "ניהול מלאי ומצאי" (21 הוראות), לא מופיע ב- `/api/Chapter` , והרשימה קופצת מ-9 ל-11.
- **11 הוראות לא מוצגות בעץ,** וחיפוש לפי המספר שלהן לא מוצא אותן. ל-7 מהן התיאור אומר שאוחדו או שמספרן שונה, כנראה במכוון.
- **סינון התגיות מת:** `/api/Tag` ריק ואין מסמך עם תגית.
- **כל הרשימה יורדת בכל ביקור** (75KB דחוסים, 1,126 מסמכים) ומסוננת בדפדפן.
- **התנהגויות API תמוהות:** מזהה קובץ שגוי מחזיר 503 עם HTML, כתובת לא קיימת מחזירה 200 עם דף הבית, קובצי Word מוגשים כ- `application/octetstream` , וכותרות ה-CORS מכריזות על פעולות כתיבה שלא אמורות להתאפשר.

לא מדדתי את ההשפעה על משתמשים אמיתיים. ההנחה שהם מבזבזים זמן או מיישמים הוראה לא נכונה היא השערה סבירה, לא מדידה.

2 הבעיה המרכזית שבחרתי לפתור: החיפוש

אדם עם מצב קונקרטי, למשל "מה עושים אם חרגנו מהתקציב?", לא מצליח להגיע בוודאות להוראה שחלה עליו. בקוד האתר (`filterByFreeTxt`) הטקסט החופשי מושווה לארבעה שדות בלבד (כותרת, תיאור, מילות מפתח, מספר) באמצעות "מכיל" פשוט: מילה חייבת להיכתב בדיוק כמו בכותרת.

מה מקלידים	מה האתר עושה
קופת ביטחון, ואחר כך קופות ביטחון	2 תוצאות ואז 4: היחיד לא מוצא את הרבים

מה מקלידים	מה האתר עושה
התחייבות, ואחר כך התחייבויות	11 תוצאות ואז 5
מקדמות ללא התחייבות	0 תוצאות, למרות שההוראות על מקדמות (1.4.5) ועל תשלומים ללא התחייבות (1.5.1) קיימות
מה עושים כשיש חריגה	0 תוצאות: שאלה היא לא כותרת
כל חיפוש בלי תוצאה	הכתובת הופכת ל- `/document/undefined` והדף נשאר ריק

למה דווקא החיפוש? כי זו הדרך שבה כמעט כל משתמש מגיע להוראה. שאר הליקויים (דגל התוקף, פרק חסר) פוגעים בדיוק, אבל כשהחיפוש לא מוצא, אין עוד מה לתקן: ההוראה הנכונה פשוט לא מגיעה אל האדם.

3 הפתרון בקצרה

אותם מסמכים, אבל החיפוש נעשה בשרת, ב-SQL, על עמודות מנוקות:

- **יחיד ורבים הם אותה מילה;** גרשיים בתוך מילה לא משנים (דו"ח = דוח); מילות שאלה (מה, האם, כשיש) מתעלמים מהן.
- **כל המילים חייבות להופיע. אם אין מסמך כזה, מוצגים הקרובים ביותר והתשובה אומרת זאת.** שאלה לא נגמרת בדף ריק.
- **דירוג:** מילה בכותרת או במספר שווה יותר ממילה בתיאור, ובשוויון הסדר הוא סדר המספרים (1.2 לפני 1.10).
- **סינון בשרת:** פרק, תת-פרק, סוג מידע ותוקף, כשהתוקף מחושב מהתאריכים ולא מהדגל ההפוך.
- **"לא נמצא" היא תשובה תקינה,** עם כתובת תקינה וכפתור לניקוי הסינון.
- **ההוראה הטובה ביותר נפתחת מיד** במסך רחב, בלי שצריך ללחוץ.

החיפוש עובד על נתוני העל של האובייקטים של האתר בלבד (כותרת, תיאור, מילות מפתח, מספר, תאריכים, פרקים). תוכן קובצי ה-Word אינו נקרא.

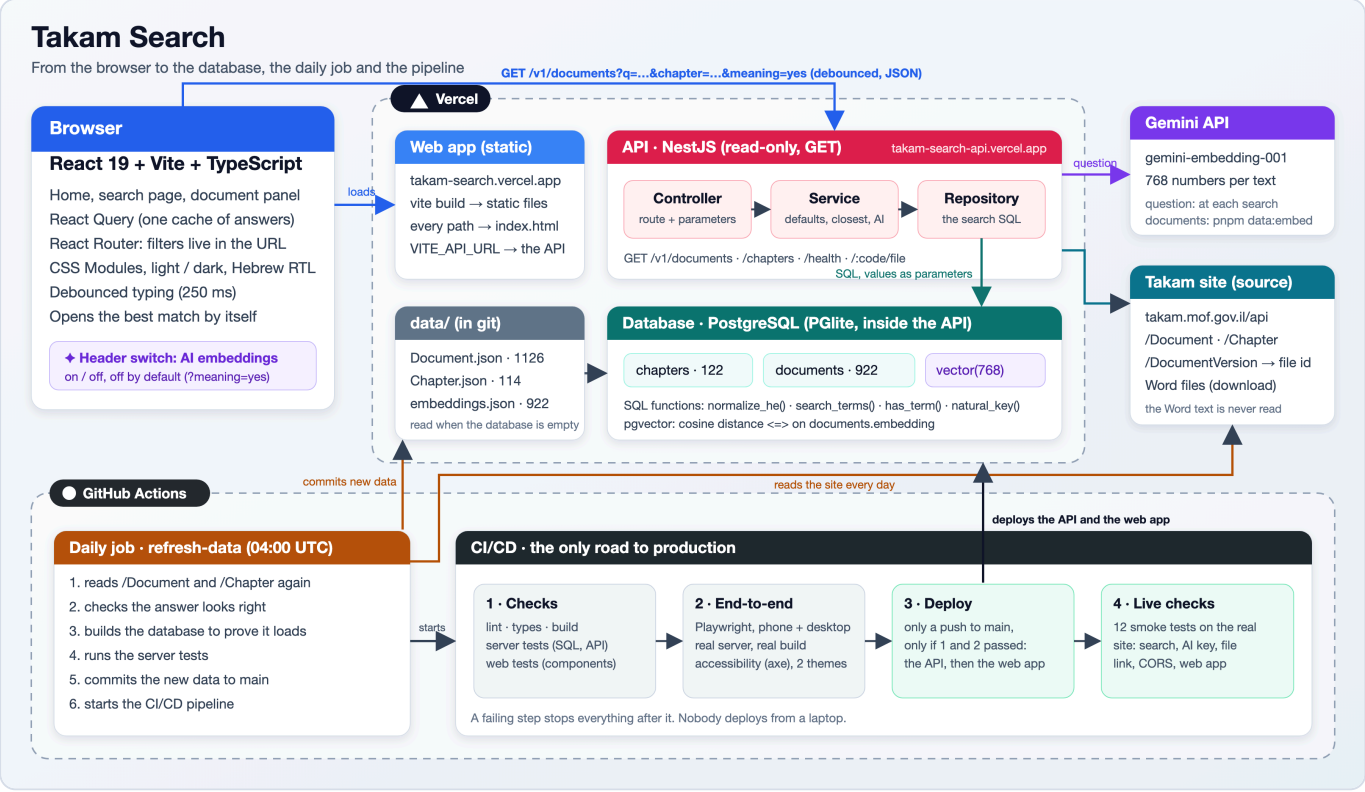
4 הארכיטקטורה: monorepo עם React, Vite, NestJS ובסיס נתונים

הפרויקט הוא `monorepo` אחד (`pnpm`), וכולו ב-`TypeScript`:

חבילה	מה יש בה
`apps/web`	**הלקוח:** `React 19 + Vite + TypeScript`. `React Query` (מטמון אחד של תשובות), `React Router` (הסינון חי בכתובת, כך שאפשר לשתף חיפוש), `CSS Modules`, עברית מימין לשמאל, מצב בהיר וכהה. מבנה הדפים כמו באתר המקורי.

חבילה	מה יש בה
`apps/server`	**השרת:** `NestJS`, תיקייה לכל route, ובכל אחת `controller → service → repository`. קריאה בלבד (`GET`).
`packages/contracts`	הטיפוסים המשותפים ללקוח ולשרת: אם צד אחד משתנה בלי השני, בדיקת הטיפוסים נכשלת.

- **בסיס הנתונים:** `PostgreSQL` אמיתי שרץ בתוך השרת (`PGlite`), בלי שום התקנה. שתי טבלאות (`documents`, `chapters`) ופונקציות SQL שמנקות עברית (`normalize_he()`, `search_terms()`). הנתונים נטענים משני קובצי JSON, האובייקטים של אתר תכ"ם, כשהבסיס ריק. החיפוש כולו הוא שאילתת SQL אחת שאפשר לקרוא.
- **המשימה היומית:** פעם ביום `GitHub Actions` קורא שוב את האתר, בודק שהתשובה נראית תקינה, מוודא שהבסיס נבנה ממנה, מריץ את בדיקות השרת, מבצע `commit` ומפעיל את ה-`CI/CD`. כך הנתונים מתעדכנים בלי יד אדם.
- **CI/CD, הדרך היחידה לפרודקשן:** בדיקות (`lint`, טיפוסים, `build`, בדיקות יחידה בשרת ובלקוח) ← בדיקות קצה-לקצה (`Playwright`, טלפון ושולחן עבודה, נגישות) ← פריסה ל-`Vercel`, רק מ-`main` ורק אחרי שהכול עבר ← 12 בדיקות על האתר החי. אף אחד לא פורס ממחשב אישי.
- **בדיקות:** 141 בשרת (ה-SQL האמיתי על הנתונים האמיתיים), 89 בלקוח ו-100 בדיקות קצה-לקצה.



מהדפדפן ועד בסיס הנתונים, כולל המשימה היומית וה-CI/CD (שמות טכניים באנגלית)

5 תוספת: חיפוש לפי משמעות, מתג שאפשר להפעיל ולכבות

בסוף הוספתי יכולת שאפשר **להפעיל או לכבות**: כפתור קטן (✦) בכותרת העליונה של דפי החיפוש. הוא כבוי

כברירת מחדל, ואז החיפוש הוא החיפוש במילים שתואר למעלה. כשמדליקים אותו, השרת מבקש מ-`Gemini` וקטור (`embedding`, 768 מספרים) לשאלה, ומשווה אותו לעמודה מסוג `pgvector` (`documents.embedding`) שיש בה וקטור לכל אחד מ-922 המסמכים, שנוצר מהכותרת, התיאור ומילות המפתח.

> **דוגמה:** השאלה "אפשר לשלם לספק לפני שקיבלתי את הסחורה". החיפוש במילים מחזיר הוראות על רכב ועל מלונות. החיפוש לפי משמעות מעלה לראש הרשימה את ההוראות 1.4.2 ו-1.4.5 (תשלום מקדמות), בלי שאף אחת מהמילים האלה מופיעה בכותרת.

- **היברידי:** מילים מדויקות ומספר הוראה ממשיכים לשקול, והמשמעות מוסיפה.
- **בטוח לכבות:** בלי מפתח `Gemini` בשרת, החיפוש לפי משמעות עונה בשגיאה ברורה (503) והחיפוש במילים לא מושפע.
- **זה ניסוי:** הוא מבוסס על נתוני העל בלבד, לא על תוכן קובצי ה-Word. השלב הבא האפשרי הוא להוסיף את תוכן המסמכים ותשובה שה-AI כותב (`RAG`) עם הפניה למקורות.